

PRACTICAL WEEK – 12

Session – 12

TASK 1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and System throughput under different buffer sizes.

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define ITEMS 10000

int *buffer;
int buffer_size;

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    int i;

    for (i = 0; i < ITEMS; i++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        in = (in + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg)
{
    int i;
    int item;
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
int i;
int item;

for (i = 0; i < ITEMS; i++)
{
    sem_wait(&full);

    pthread_mutex_lock(&mutex);

    item = buffer[out];
    out = (out + 1) % buffer_size;

    pthread_mutex_unlock(&mutex);

    sem_post(&empty);
}

return NULL;
}

void run_test(int size)
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    struct timespec start, end;

    buffer_size = size;

    buffer = malloc(buffer_size * sizeof(int));

    if (buffer == NULL)
    {
        perror("malloc");
        exit(1);
    }

    in = 0;
    out = 0;

    sem_init(&empty, 0, buffer_size);
    sem_init(&full, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    clock_gettime(CLOCK_MONOTONIC, &start);

    pthread_create(&producer_thread, NULL,
                  producer, NULL);

    pthread_create(&consumer_thread, NULL,
                  consumer, NULL);
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1

pthread_create(&producer_thread, NULL,
               producer, NULL);

pthread_create(&consumer_thread, NULL,
               consumer, NULL);

pthread_join(producer_thread, NULL);
pthread_join(consumer_thread, NULL);

clock_gettime(CLOCK_MONOTONIC, &end);

double time_taken =
    (end.tv_sec - start.tv_sec) +
    (end.tv_nsec - start.tv_nsec) / 1000000000.0;

printf("Buffer Size : %d\n", buffer_size);
printf("Items      : %d\n", ITEMS);
printf("Time       : %.6f seconds\n", time_taken);

if (time_taken > 0)
{
    printf("Throughput : %.2f items/second\n",
           ITEMS / time_taken);
}

printf("-----\n");

sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

free(buffer);
}

int main()
{
    printf("Session 11 - Producer Consumer\n");
    printf("-----\n");

    printf("\nTesting different buffer sizes:\n\n");

    run_test(2);
    run_test(5);
    run_test(10);
    run_test(20);

    printf("Producer-Consumer test completed successfully.\n");

    return 0;
}
```

```

keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ nano session12_task1.c
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ gcc session12_task1.c -o session12_task1 -pthread
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./session12_task1
Session 11 - Producer Consumer

-----
Testing different buffer sizes:
Buffer Size : 2
Items       : 10000
Time        : 0.291741 seconds
Throughput  : 34277.02 items/second
-----
Buffer Size : 5
Items       : 10000
Time        : 0.130277 seconds
Throughput  : 76759.37 items/second
-----
Buffer Size : 10
Items       : 10000
Time        : 0.052214 seconds
Throughput  : 191520.95 items/second
-----
Buffer Size : 20
Items       : 10000
Time        : 0.021646 seconds
Throughput  : 461973.83 items/second
-----
Producer-Consumer test completed successfully.
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ |

```

TASK 2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```

GNU nano 8.7.1
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;

/* Deadlock demonstration */
void *thread1_deadlock(void *arg)
{
    printf("Thread 1: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 1: Waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2_deadlock(void *arg)
{
    printf("Thread 2: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    sleep(1);

    printf("Thread 2: Waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Acquired both resources\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

/* Deadlock prevention using resource ordering */
void *thread1_ordered(void *arg)

```

```

/* Deadlock prevention using resource ordering */
void *thread1_ordered(void *arg)
{
    printf("Thread 1: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Both resources acquired safely\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2_ordered(void *arg)
{
    printf("Thread 2: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Both resources acquired safely\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    printf("Session 12 - Deadlock Demonstration\n");
    printf("-----\n\n");

    printf("Deadlock prevention using resource ordering.\n\n");
    printf("Starting ordered threads...\n\n");

    pthread_create(&t1, NULL, thread1_ordered, NULL);
    pthread_create(&t2, NULL, thread2_ordered, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\nBoth threads completed successfully.\n");
    printf("Deadlock prevented using resource ordering.\n");

    return 0;
}

```

```

keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ nano session12_task2.c
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ gcc session12_task2.c -o session12_task2 -pthread
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./session12_task2
Session 12 - Deadlock Demonstration
-----

Deadlock prevention using resource ordering.

Starting ordered threads...

Thread 1: Locking Resource 1
Thread 1: Locking Resource 2
Thread 1: Both resources acquired safely
Thread 2: Locking Resource 1
Thread 2: Locking Resource 2
Thread 2: Both resources acquired safely

Both threads completed successfully.
Deadlock prevented using resource ordering.
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ |

```

